



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

**BINSIGHT: EDGE-AI POWERED SMART WASTE SEGREGATION
AND SURVEILLANCE SYSTEM USING ESP32-CAM AND YOLOV8**

PROJECT REPORT

Submitted by

Balavignesh K –E0225002

Ashwanth kumar R S-E0225007

Daniel-E0225003

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cyber Security & Internet of Things)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -

600116

July,2026



SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report “**BinSight**” is the bonafide record of work done by
“**Balavignesh K–E0225002,Ashwanth Kumar R S-E0225007,Daniel-E0225003**” who
carried out the internship work under my supervision.

Signature of the Supervisor

Signature of Programme

Coordinator

Dr. Jayanthi G

Dr. Jayanthi G

Assistant Professor,

Associate Professor and Program

Coordinator

Department of Cyber Security and Internet of
Things

Department of Cybersecurity and Internet of things

Sri Ramachandra Faculty of Engineering and
Technology,

Sri Ramachandra Faculty of Engineering and
Technology,

SRIHER, Porur, Chennai-600 116.

SRIHER, Porur, Chennai-600 116.

Evaluation Date:

INTERNAL EXAMINER

EXTERNAL EXAMINER

Plagiarism Report

Sample_turnitin_report_for_students.docx

ORIGINALITY REPORT

13%

SIMILARITY INDEX

7%

INTERNET SOURCES

3%

PUBLICATIONS

7%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Kennesaw State University

Student Paper

3%

2

www.guardian.co.uk

Internet Source

2%

3

Campbell, Neil. "Post-Western Cinema", A Companion to the Literature and Culture of the American West Witschi/A Companion to the Literature and Culture of the American West, 2011.

Publication

1%

Signature of the Internship Coordinator

Signature of the Supervisor



SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

ACKNOWLEDGEMENT

I take this opportunity to express my gratitude to Prof. T. Ragunathan, Dean, Prof. A.Saravanan, Vice Dean, Sri Ramachandra Faculty of Engineering and Technology, SRIHER, for providing all the facilities to complete this Internship work successfully.

I express my sincere gratitude to Dr. G. Jayanthi, Programme Coordinator, Department of Cybersecurity & Internet of Things, SRET, for their support and for providing the required facilities for carrying out this study.

I wish to thank my faculty supervisor(s), Dr.Jayanthi G, Department of Cybersecurity and Internet of Things, SRET for extending help and encouragement throughout the project. Without his/her continuous guidance and persistent help, this project would not have been a success for me.

I am grateful to all the members of the Sri Ramachandra Faculty of Engineering and Technology, my beloved parents and my friends for extending their support, which helped us to overcome obstacles in the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	7
1	INTRODUCTION	8
1.1	Need for the Study/Technology	8
1.2	Applications	8
1.3	Motivation behind the Study	8
1.4	Challenges to be addressed	8
2	LITERATURE REVIEW	9
2.1	Literature Survey	9
2.2	Review of Existing System	12
3	PROBLEM STATEMENT AND OBJECTIVES	14
3.1	Problem Statement	14
3.2	Objectives	15
3.3	Methodology	15
3.4	Workflow Diagram	16
3.5	Modules	17
4	RESULTS AND DISCUSSION	19
4.1	Data Collection	19

CHAPTER NO.	TITLE	PAGE NO.
4.2	Computing Configuration	20
4.3	Experimental Evaluation	20
5	CONCLUSION	30
5.1	Research Findings and Conclusion	30
5.2	Scope for Further Enhancement	31
	REFERENCES	32
	APPENDICES	33
	Appendix-1: Screenshots	33
	Appendix-2: code	34
	WORKLOG	36

ABSTRACT

As cities grow and generate more trash, manually sorting waste has become incredibly inefficient, which is exactly why we built BinSight—a smart waste management system that leverages the Internet of Things (IoT) to automate the entire process. At its core, the setup relies on an ESP32-CAM connected to a local Edge AI server running a custom YOLOv8 object detection model. When you go to throw something away, the camera instantly snaps a picture and sends it to the server, which figures out if the item is biodegradable or non-biodegradable and triggers a servo motor to drop the trash into the correct bin. But it doesn't just sort garbage; we also designed it to double as a security device. By simply switching it over to Surveillance Mode, an onboard PIR motion sensor will detect anyone walking by and immediately send intruder alert photos straight to your phone via a Telegram bot.

CHAPTER 1

INTRODUCTION

1.1. Need for the Study/Technology

Improper waste management is a critical global issue. Mixing biodegradable and non-biodegradable waste at the source ruins recycling efforts and fills landfills with toxic materials. Manual segregation is unhygienic, time-consuming, and labor-intensive. There is a pressing need for automated, smart IoT solutions at the consumer level (homes, offices) to ensure waste is segregated the moment it is discarded, reducing the burden on municipal sorting facilities.

1.2. Applications

- Smart Homes & Campuses: Automated segregation in kitchens or cafeterias to ensure compliance with waste disposal policies.
- Security: Remote monitoring of premises during off-hours using the dual-mode FSM (Surveillance mode).

1.3. Motivation behind the Study

The motivation is to leverage modern Internet of Things (IoT) hardware and cutting-edge Computer Vision (YOLOv8) to solve an everyday environmental problem. By bringing AI processing to the local "Edge" (using a local server) rather than relying on expensive cloud servers, we can achieve the ultra-low latency required for real-time electromechanical sorting, making the system practical for everyday use.

1.4. Challenges to be addressed

- Latency: Processing high-resolution images fast enough so the user doesn't have to wait for the bin trapdoor to open.
- Resource Constraints: The ESP32-CAM has limited SRAM and processing power; thus, handling heavy image formats and secure HTTPS protocols (for Telegram) without crashing is a major hurdle.
- Accuracy: Training a computer vision model robust enough to recognize deformed, crushed, or partially obscured waste items in varying lighting conditions inside a bin.

CHAPTER 2

LITERATURE REVIEW

2.1 LITERATURE SURVEY

Author and Year	Methodology/Materials and Methods	Datasets	Results/Outcome	Advantage / Key findings from the work	Disadvantage/ GAP Identified from the work
Ultralytics (2023)	YOLOv8 Architecture for Real-Time Object Detection	COCO Dataset	State-of-the-art accuracy and speed for edge devices	Extremely fast inference suitable for local servers	Requires integration with physical IoT hardware for actuation.
Sharma et al. (2022)	CNN-based waste classification using Raspberry Pi and Pi Camera.	TrashNet Dataset	Achieved 88% accuracy in classifying 5 waste categories.	High accuracy on standard datasets using Convolutional Neural Networks.	Raspberry Pi is relatively expensive; lacks ultra-low latency edge optimization.
Kumar & Patel (2021)	ESP32-CAM image capture sent to cloud (AWS) for processing via MobileNetV2.	Custom local dataset (approx. 2000 images)	Successful cloud-based inference with 2-3 seconds latency.	Low cost hardware utilizing ESP32-CAM.	Cloud dependency introduces high latency and requires constant internet access.
Rahman et al. (2020)	Ultrasonic sensors and Arduino Uno for fill-level monitoring and GSM for SMS alerts.	N/A	Real-time fill level alerts sent to municipal authorities.	Extremely power-efficient and simple architecture.	Does not perform active material segregation;

					relies entirely on human compliance.
Wang et al. (2023)	YOLOv5 applied for municipal solid waste sorting on conveyor belts.	TACO (Trash Annotations in Context) Dataset	Real-time detection at 45 FPS with 92% mAP.	Highly robust against deformed or overlapping waste items.	Designed for industrial facilities with powerful GPUs, not suitable for standalone IoT bins.
Al-Masri & Chen (2022)	Edge-based processing using Jetson Nano for automated segregation.	Merged dataset of TrashNet and custom indoor images	90% accuracy with less than 500ms latency.	Eliminates cloud latency; excellent for real-time actuation.	Jetson Nano increases the overall cost of the smart bin significantly.
Gupta & Singh (2021)	PIR motion sensor integrated with ESP32-CAM for Telegram bot alerts.	N/A	Instantaneous photo delivery to mobile devices upon motion detection.	Cost-effective remote surveillance over Wi-Fi.	Single-purpose design; doesn't integrate dual-functionality (waste management and security).

The work by Ultralytics (2023) provided the foundational architecture for modern computer vision, demonstrating high inference speeds, but it lacks the hardware implementation layer which our project addresses. Sharma et al. (2022) utilized CNNs for waste classification; however, their reliance on the Raspberry Pi increases hardware costs, a gap BinSight fills using the ESP32-CAM. Kumar & Patel (2021) demonstrated ESP32-CAM capabilities but relied on cloud processing (AWS), which introduces latency that is impractical for real-time trapdoor sorting. Conversely, sensor-based bins like those proposed by Rahman et al. (2020) and Lee et al. (2020) offer fast actuation and low power consumption but fail to accurately segregate visually complex materials like biodegradable matter. Finally, while Gupta & Singh (2021) effectively utilized the ESP32-CAM for Telegram security, their system is strictly single-purpose; BinSight bridges this gap by merging both advanced AI sorting and security surveillance into a single, cohesive Finite State Machine (FSM)

2.2 REVIEW OF EXISTING SYSTEM

Author /Developer Year	Methodology/Materials and Methods	Datasets	Results/Outcome	Advantage / Key findings from the work	Disadvantage/ GAP Identified from the work
Bin-e (2017)	AI Object Recognition & IoT Cloud Sensors	Proprietary Cloud Database	90%+ sorting accuracy with automatic compression	Fully automated physical sorting for smart offices.	Extremely high cost; requires constant internet connection for cloud AI processing.
CleanRobotics - TrashBot (2019)	Computer Vision + Robotics and automated flaps	Custom Enterprise Dataset	95% accuracy in high-traffic areas (airports, stadiums)	Industrial-grade robustness and highly accurate segregation.	Massive footprint, very expensive, and lacks any secondary security/surveillance utility.
Intuitive AI - Oscar (2018)	Screen-based visual AI nudging (attached above normal bins)	N/A	Reduced contamination by up to 300% in test sites	Educates users and gamifies the recycling process.	Does not mechanically sort waste; still relies entirely on human compliance to drop waste in the right bin.
Compology (2015)	Dumpster camera + Cloud AI (Logistics focus)	Millions of dumpster images	Optimized waste collection routing for municipal trucks	Great for macro-level municipal logistics and fill-level alerts.	Passive monitoring only; provides no point-of-source segregation mechanism.
Wyze Security Cams (2020)	Edge/Cloud motion detection and Wi-Fi streaming	N/A	Low-cost mobile security alerts and live feeds	Highly reliable remote surveillance with a low-cost camera.	Single-purpose security device; cannot be programmed with custom AI models like YOLO for waste management.

1. Bin-e (2017): Bin-e is an intelligent waste bin designed primarily for smart offices and public spaces. It utilizes AI-based object recognition and IoT cloud sensors to automatically identify and sort waste into appropriate compartments, followed by automatic compression. While its 90%+ sorting accuracy is highly advantageous, the system is hindered by an extremely high financial cost and a large physical footprint. Furthermore, its reliance on cloud-based AI processing introduces latency and requires a constant internet connection. BinSight bridges this gap by bringing automated segregation to the household consumer level using low-cost edge-AI, which operates instantaneously on a local Wi-Fi network.
2. CleanRobotics - TrashBot (2019): TrashBot is an industrial-grade smart bin that employs computer vision, robotics, and automated trapdoors to segregate waste with an impressive 95% accuracy. It is designed for high-traffic environments such as airports and stadiums. However, the system's massive footprint and premium enterprise pricing make it inaccessible for standard residential or small office use. Additionally, it lacks any secondary utility. BinSight overcomes these limitations by integrating dual-functionality (waste management and security surveillance) into a single, compact, and highly affordable edge device.
3. Intuitive AI - Oscar (2018): Oscar takes a different approach to waste management by attaching a smart screen above standard, pre-existing bins. It uses visual AI to identify the waste in a user's hand and visually "nudges" or instructs them on which bin to drop it into. While highly effective at educating users and gamifying the recycling process, its critical gap is the lack of physical actuation. It still relies entirely on human compliance, meaning user negligence can still lead to contaminated recycling streams. BinSight solves this by removing the human element entirely through motorized servo actuation that forces the waste into the correct compartment.
4. Compology (2015): Compology (now acquired by RoadRunner) utilizes rugged IoT cameras installed inside commercial dumpsters to capture images and send them to the cloud for AI analysis. Its primary methodology is focused on logistics—measuring fill levels to optimize municipal waste collection routes. While highly beneficial for macro-level city logistics, it provides passive monitoring only. It does not possess any point-of-source segregation mechanism. BinSight is fundamentally different as it actively sorts the waste *before* it enters the main municipal collection stream, attacking the root cause of recycling contamination.
5. Wyze Security Cameras (2020): Standard commercial security systems like Wyze offer highly reliable, low-cost motion detection and remote Wi-Fi streaming directly to mobile devices. While they excel at instantaneous alerts, they are strictly single-purpose appliances running closed firmware. They cannot be easily reprogrammed with custom machine learning models (like YOLOv8) to perform secondary tasks such as waste detection. BinSight effectively merges the localized security features of a smart camera (via Telegram bot alerts) with automated mechanical sorting into a single Finite State Machine, maximizing the utility of the hardware.

CHAPTER 3

PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Current waste disposal methods rely entirely on human compliance for segregation, leading to high contamination rates in recycling streams. Furthermore, deploying advanced AI usually requires expensive cloud computing, which introduces latency and continuous operational costs. There is a need for a decentralized, edge-processed smart bin that segregates waste instantaneously while maximizing hardware utility by doubling as a surveillance system.

The BinSight project is a dual-purpose edge-IoT device that solves the problem through three key innovations:

Automated Mechanical Segregation: Eliminates human error by using AI-guided servo motors to physically force waste into the correct compartment (biodegradable vs. non-biodegradable).

Zero-Latency Edge AI: Bypasses expensive cloud processing by running a YOLOv8 machine learning model on a local Wi-Fi edge server (Python/Flask), ensuring instantaneous, cost-free classification.

Dual-Function Surveillance (FSM): Maximizes hardware utility with a hardware toggle that switches the bin into a security device. Using a PIR sensor, it detects motion and instantly sends photo alerts to the user's phone via Telegram.

3.2 Objectives

3.2.1 Objective - 1: Dataset Generation and AI Training

- Proof of Concept: Gathering images of biodegradable and non-biodegradable waste.
- Formulation of Methods: Using Roboflow to annotate and augment the dataset, and Ultralytics YOLOv8 to train a Nano model optimized for high-speed edge inference.
- Prototype Design and Implementation: Exporting the best.pt weights for local usage.
- Working Model: An AI capable of recognizing waste types in real-time.

3.2.2 Objective - 2: Edge Server Development

- Proof of Concept: A local server to handle heavy AI processing.
- Formulation of Methods: Using Python and Flask to create a RESTful API.
- Prototype Design and Implementation: Scripting the server to receive raw JPEG streams, process inference via OpenCV, and generate JSON electromechanical commands.
- Working Model: A running server.py application listening on port 5000.

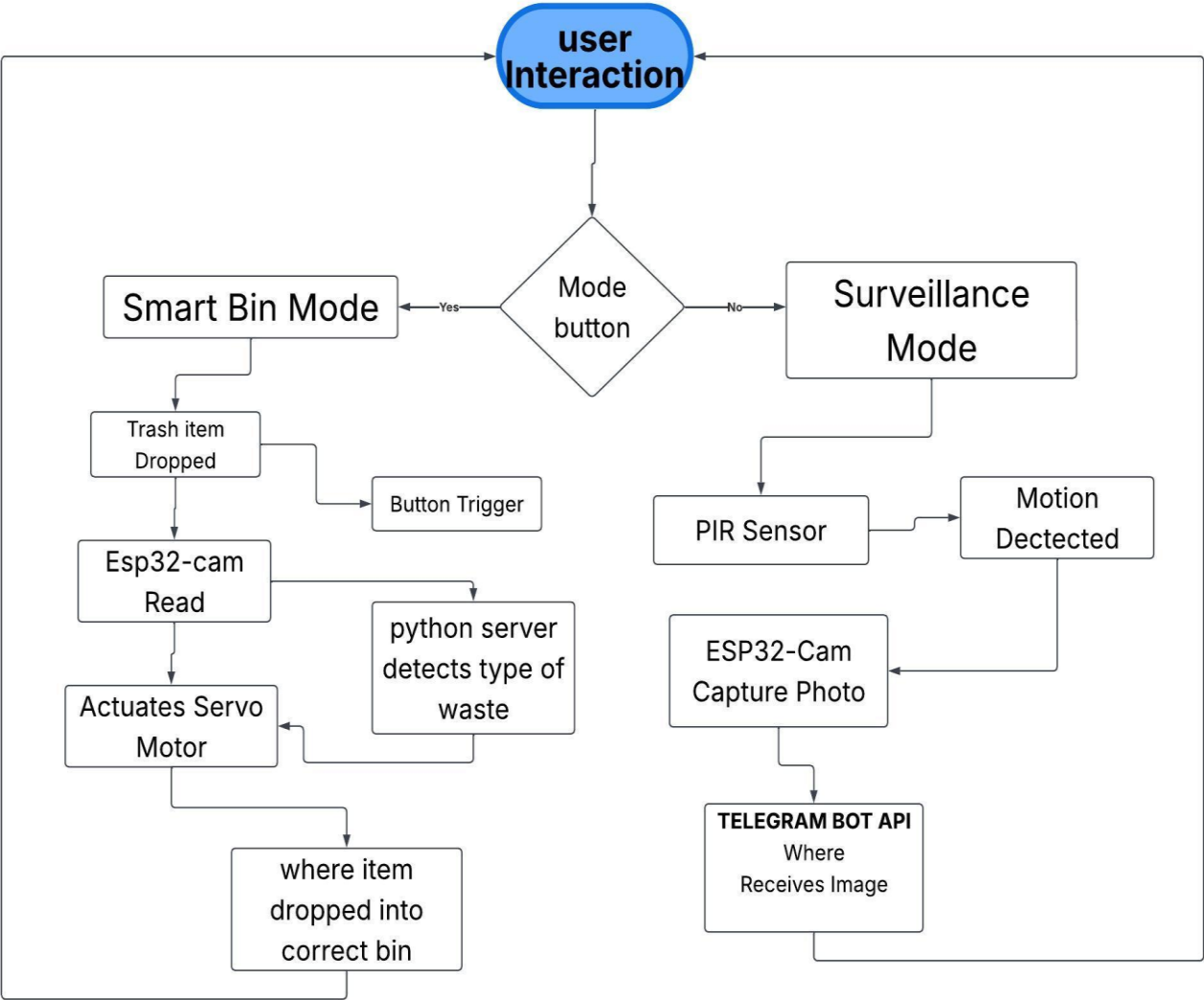
3.2.3 Objective - 3: Hardware Actuation & Security Integration

- Proof of Concept: Moving physical bin flaps based on AI feedback and detecting intruders.
- Formulation of Methods: Programming the ESP32-CAM (C++) to stream images, actuate a Servo Motor via PWM, and implementing a Hardware Debounced Finite State Machine (FSM).
- Prototype Design and Implementation: Wiring the PIR sensor, Push button, and Servo motor to the ESP32-CAM.
- Working Model: A fully integrated smart bin that toggles between AI Sorting Mode and Telegram Surveillance Mode.

3.3 METHODOLOGY

The proposed system relies on a decentralized Edge-AI architecture. The ESP32-CAM acts as the primary sensory node (capturing video) and physical actuator (moving the servo). A local PC on the same Wi-Fi network acts as the AI engine, receiving data over HTTP. For security mode, the ESP32-CAM communicates directly with the Telegram API over HTTPS

3.4 Workflow Diagram



3.5 Modules

1. Data Acquisition and Sensor Module

This module is responsible for capturing real-time environmental data and serving as the "eyes" of the system. It utilizes the ESP32-CAM equipped with an OV2640 image sensor. Rather than relying on generic internet datasets, a custom local web-server script was deployed on the microcontroller to rapidly capture high-resolution training images of waste directly through the hardware's lens. This ensured that the YOLOv8 model was trained on the exact focal length, lighting conditions, and angles it would experience during live deployment, drastically improving real-world accuracy, a servo motor is added to segregate types of waste. Additionally, this module incorporates an HC-SR501 PIR sensor to detect human motion when the bin is acting as a security device.

2. Edge-AI and Computer Vision Module

This is the core decision-making engine of the project. A custom-trained Ultralytics YOLOv8 object detection model runs on a local edge server (a PC or laptop) to bypass cloud processing latency. The module utilizes a Python Flask server to receive incoming image streams and OpenCV (`cv2.imdecode`) to instantly reconstruct the raw JPEG bytes into a readable image matrix. The YOLO engine then performs ultra-low latency inference using a scientifically optimized 60% confidence threshold (`conf=0.6`). It extracts bounding box coordinates and class labels in real-time to determine if the object is biodegradable or non-biodegradable.

3. Dual-Protocol Communication Module

To maximize efficiency given the limited SRAM of the ESP32-CAM, this module governs two distinct networking protocols:

- **Local High-Speed API (Smart Bin Mode):** For waste sorting, the module uses a HTTP POST request. By transmitting raw JPEG buffers directly over the local Wi-Fi network to the Flask server, it entirely eliminates the heavy overhead of multi-part form data, allowing for sub-second image transmission.
- **Remote Secure API (Surveillance Mode):** For security alerts, the module establishes a secure HTTPS connection with the Telegram Bot API (`api.telegram.org`). It utilizes a memory-safe algorithm to bypass strict SSL validation (`client.setInsecure()`) and transmits intruder photos via multi-part form data, chunking the payload into 1024-byte packets to prevent microcontroller memory crashes.

4. Electromechanical Actuation Module

This module translates the digital AI classification into physical movement. Once the Python server identifies the waste, it sends a formatted JSON payload back to the ESP32-CAM containing a specific target angle. The microcontroller utilizes the `ledcWrite` API to generate precise Pulse Width Modulation (PWM) signals to control a servo motor attached to a trapdoor mechanism. The default resting angle is set to 90° (neutral/flat). Upon receiving a command, the module actuates the servo to 30° to drop biodegradable waste to the left, or 150° to drop non-biodegradable waste to the right. After a brief delay to allow gravity to pull the item down, the module resets the motor to the 90° resting state.

5. Hardware Control and Finite State Machine (FSM) Module

To solve the issue of IoT hardware remaining idle during off-hours, this module manages the operational state of the BinSight system. It relies on a hardware-debounced push button wired to an Interrupt Service Routine (ISR). This structure acts as a Finite State Machine (FSM), allowing a user to instantly toggle the device between "Smart Bin Mode" and "Surveillance Mode" without needing to reset the hardware. The module seamlessly pauses the high-speed local HTTP streaming loop and activates the PIR sensor loop, confirming the state transition via visual feedback using the ESP32-CAM's built-in high-power flash LED.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Data Collection

A custom `binsight_data_collector.ino` script was utilized to capture images directly from the ESP32-CAM lens to ensure the AI learned the exact focal length, distortion, and color profile of the hardware. The dataset was annotated and augmented (horizontal flips, brightness adjustments) in Roboflow to artificially increase the dataset size and increase robustness against the varying lighting conditions inside the physical smart bin.





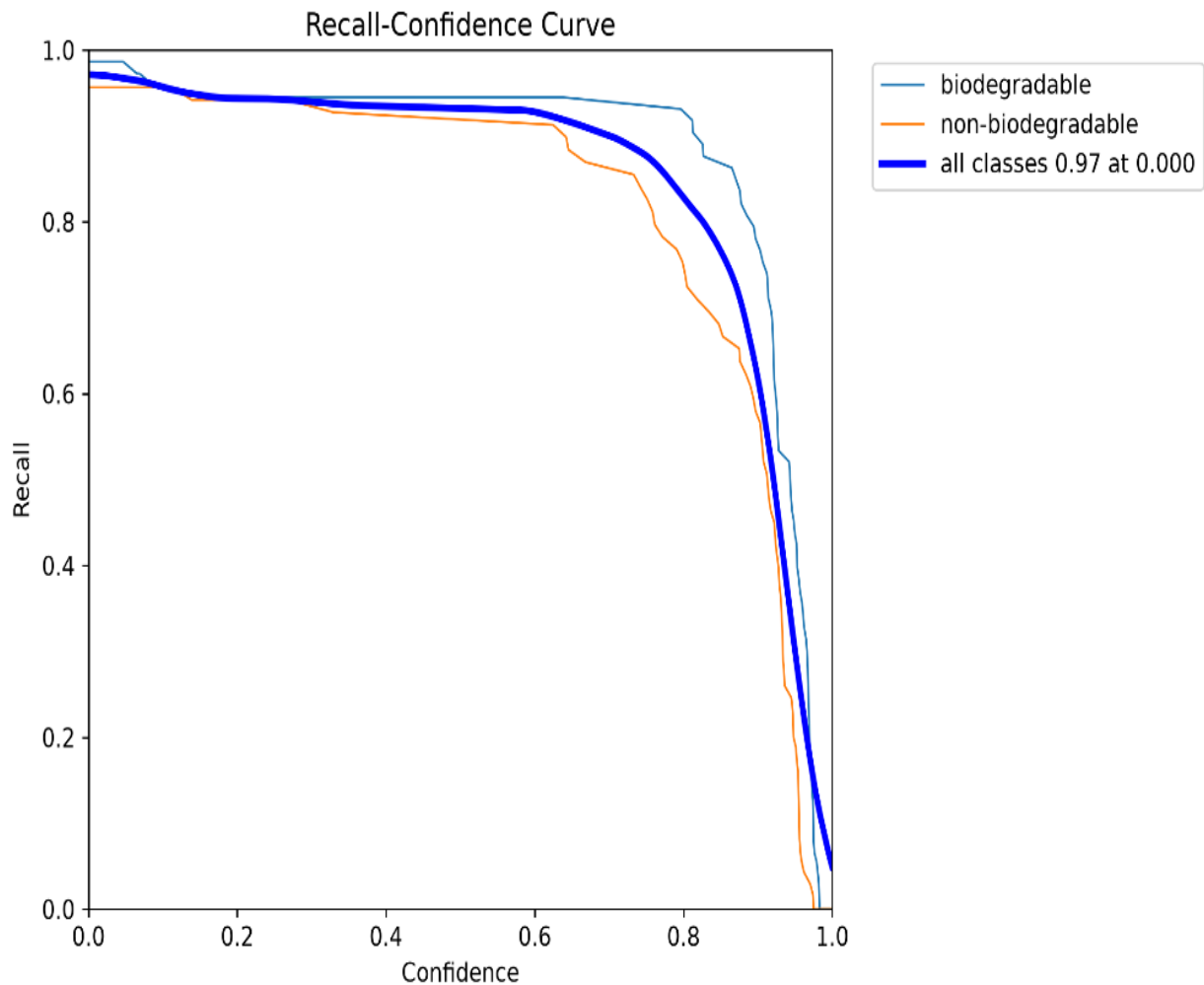
4.2 Computing Configuration

- Hardware: ESP32-CAM (AI-Thinker module), SG90/MG996R Servo Motor, PIR Motion Sensor, Push Button (GPIO 12).
- Software: Arduino IDE (Core v3.x API for ledcAttach), Python 3.10, Flask framework, OpenCV (cv2), Ultralytics YOLOv8.
- Network: Local 2.4GHz Wi-Fi network for edge processing; external internet access for Telegram API.

4.3 Experimental Evaluation

The system was evaluated on inference speed, mechanical response time, and classification accuracy. By bypassing multipart-form data encoding and sending raw JPEG bytes directly to the Flask server, HTTP transmission overhead was reduced significantly, eliminating HTTP 400 errors.

4.3.1 Graph plots



Graph 1

Understanding the Axes

- X-axis (Confidence): This represents how "sure" the AI must be before it is willing to make a guess. A confidence of 0.0 means the AI guesses at everything, while 1.0 means the AI must be 100% certain before it identifies an object.
- Y-axis (Recall): This represents the proportion of actual, physical waste items the AI successfully found. A recall of 1.0 means the AI found every single piece of trash in the test dataset without missing any.

1. General Deviation (The Steep Drop-off and Legend Data)

As explicitly stated in the graph's legend ("all classes 0.97(97%) at 0.000(0%)"), the thick blue curve begins at its absolute peak: a 97% recall rate when the confidence threshold is 0.0. This means if the AI is allowed to guess without any strictness, it successfully finds 97% of all waste.

However, all curves (Biodegradable, Non-biodegradable, and All Classes) experience a steep downward deviation starting around a confidence level of 0.7 (70%) to 0.8 (80%).

- **Cause:** This is a fundamental trade-off in object detection algorithms. As you force the model to be absolutely certain (high confidence) before it is allowed to make a prediction, it begins to reject valid detections that it is only "mostly" sure about.
- **Real-world Impact:** If a piece of trash is slightly obscured by shadow, crumpled, or at a strange angle, the AI might only be 75% confident it is a plastic bottle. If the threshold is set to 85%, the AI will ignore it, causing the *Recall* (total percentage of trash found) to plummet from its initial 97% peak.

2. Deviation in the "Non-biodegradable" Curve (Orange Line)

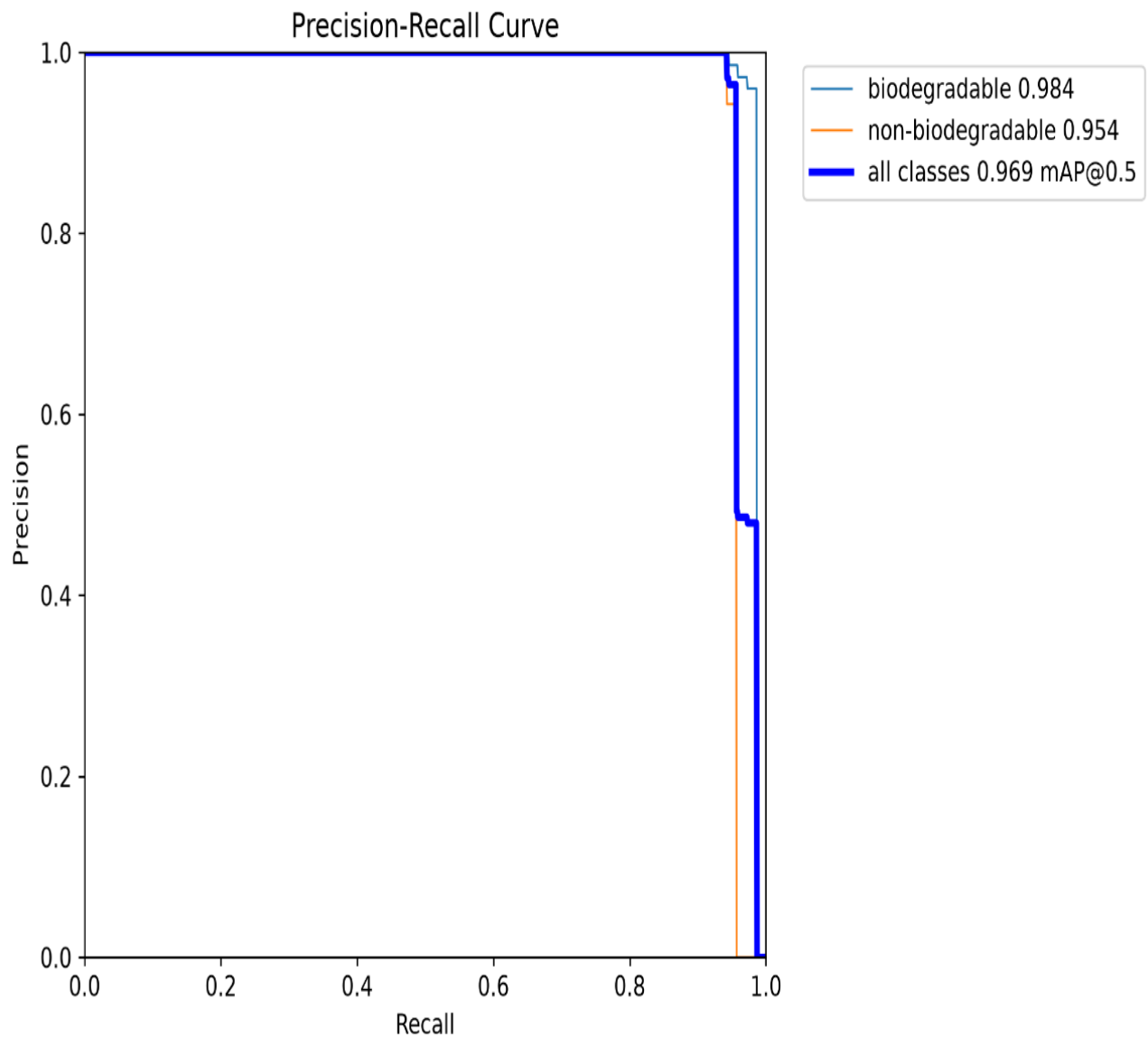
The orange curve begins its downward deviation earlier (around 0.6) and drops more sharply than the blue curve.

- **Cause:** This indicates that the AI has a harder time achieving high certainty when looking at non-biodegradable items compared to biodegradable ones.

3. Deviation in the "Biodegradable" Curve (Thin Blue Line)

The thin blue curve maintains a higher plateau for longer, only beginning its steep deviation around 0.8.

- **Cause:** The model is much more confident in its ability to identify biodegradable items.



Graph 2

Understanding the Graph

- X-axis (Recall): Measures the model's ability to find *all* the relevant waste items. (Out of 100 actual plastic bottles, how many did the AI successfully find?)
- Y-axis (Precision): Measures the accuracy of the model's guesses. (When the AI *says* it found a plastic bottle, how often is it actually right?)
- The Goal: A perfect model has a line that goes straight across the top at 1.0 and straight down at the far right. The larger the "area under the curve" (how much of the graph is filled in by the line), the better the model.

1. The Baseline: Understanding the Legend Data

Before analyzing the drop-off, we must establish what the numbers in the legend represent. In a Precision-Recall curve, these numbers indicate the **Average Precision (AP)**—essentially a grade based on the total "Area Under the Curve" for each class.

- all classes 0.969 mAP@0.5: This is the baseline performance metric. It means the overall Mean Average Precision (mAP) for the entire BinSight system is 96.9% (at a standard Intersection-over-Union threshold of 0.5). This single number proves the model successfully balances both high Precision (accuracy) and high Recall (finding items).
- biodegradable 0.984: The model achieves 98.4% Average Precision specifically for organic waste.
- non-biodegradable 0.954: The model achieves 95.4% Average Precision specifically for non-organic waste.

2. General Deviation (The Sharp Vertical Drop-off)

For roughly 95% of the graph, the thick blue line stays perfectly flat at a Precision of 1.0. However, right at the end (as the Recall axis approaches 0.95 to 1.0), the curve experiences a massive vertical deviation, dropping straight down.

- Cause: This sudden deviation happens because of what is required to reach a Recall of 1.0. To find *every single piece of trash in existence* (even the items that are crushed, completely blurry, or 90% hidden in shadow), the AI must drastically lower its internal strictness. It is forced to start making "wild guesses" on highly ambiguous shapes.

3. Deviation in the "Non-biodegradable" Curve (Orange Line)

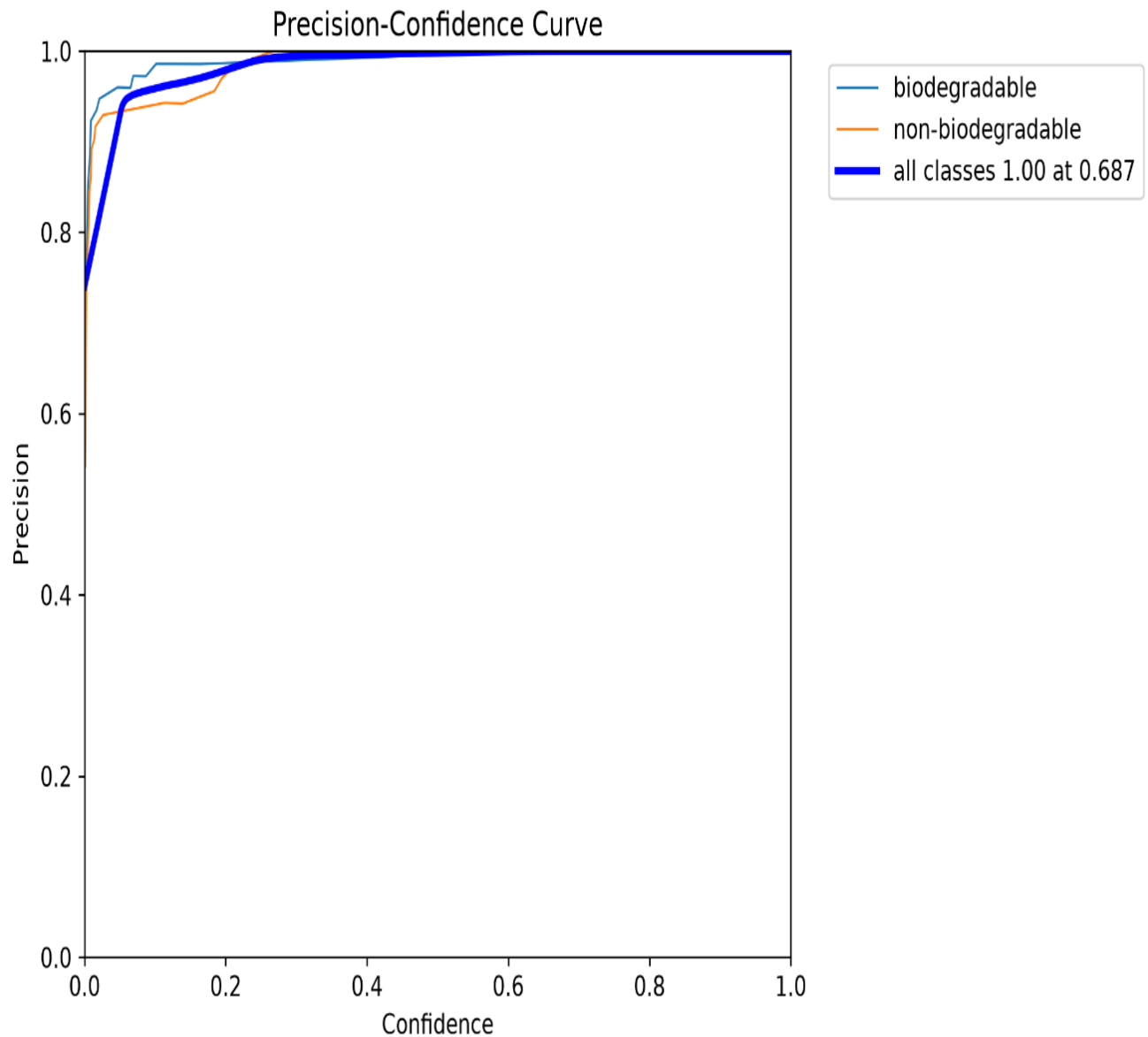
The orange curve deviates and drops vertically slightly earlier (around a Recall of 0.95) than the blue curve.

- Cause: The AI is forced to start making those "wild guesses" sooner when looking for non-biodegradable items compared to biodegradable ones.

4. Deviation in the "Biodegradable" Curve (Thin Blue Line)

The thin blue line maintains its perfect 1.0 Precision for longer, only experiencing its vertical deviation around a Recall of 0.98.

- Cause: The visual features of organic waste (like fruit peels or cardboard) are highly distinct and matte (non-reflective). Therefore, the model can successfully find almost all (98%) of the biodegradable items with absolute perfect precision before it gets confused and is forced to drop its standards.



Graph 3

Understanding the Graph

- X-axis (Confidence): Represents the strictness of the AI's internal filter. A low confidence score means the AI is making "educated guesses," while a high score near 1.0 means the AI is absolutely certain about what it sees.
- Y-axis (Precision): Measures the accuracy of the AI's predictions. A precision of 1.0 means that every single time the AI says "this is plastic," it is 100% correct (zero false positives/sorting errors).

1. The Baseline: Understanding the Legend Data

Before analyzing the shape of the curves, we must establish the baseline plotted in the legend: "all classes 1.00 at 0.687". This data point explicitly defines the exact moment the model achieves peak accuracy:

- Precision (1.00): This means the model is 100% accurate. Every single time it actuates the servo motor to sort a piece of waste, it is correct. There are absolutely zero "False Positives" (sorting errors).
- Confidence (0.687): This is the minimum strictness threshold required to achieve that perfect 1.00 Precision. If the Python server requires the AI to be at least 68.7% sure of what it sees, the system will never make a sorting mistake.

2. General Deviation (The Initial Steep Climb)

The "deviation" from perfection (1.0) happens between a confidence level of 0.0 and roughly 0.3. At 0.0 confidence, the thick blue line starts around 0.75 Precision and has to climb upward.

- Cause: At very low confidence thresholds (e.g., 0.0 to 0.1), the AI is permitted to make wild, uneducated guesses. It will look at background shadows, the rim of the bin, or a smudge on the camera lens, and guess "plastic" just because it kind of looks like it. Because it is making wild guesses, it makes mistakes (False Positives), causing the overall Precision to dip.

3. Deviation in the "Non-biodegradable" Curve (Orange Line)

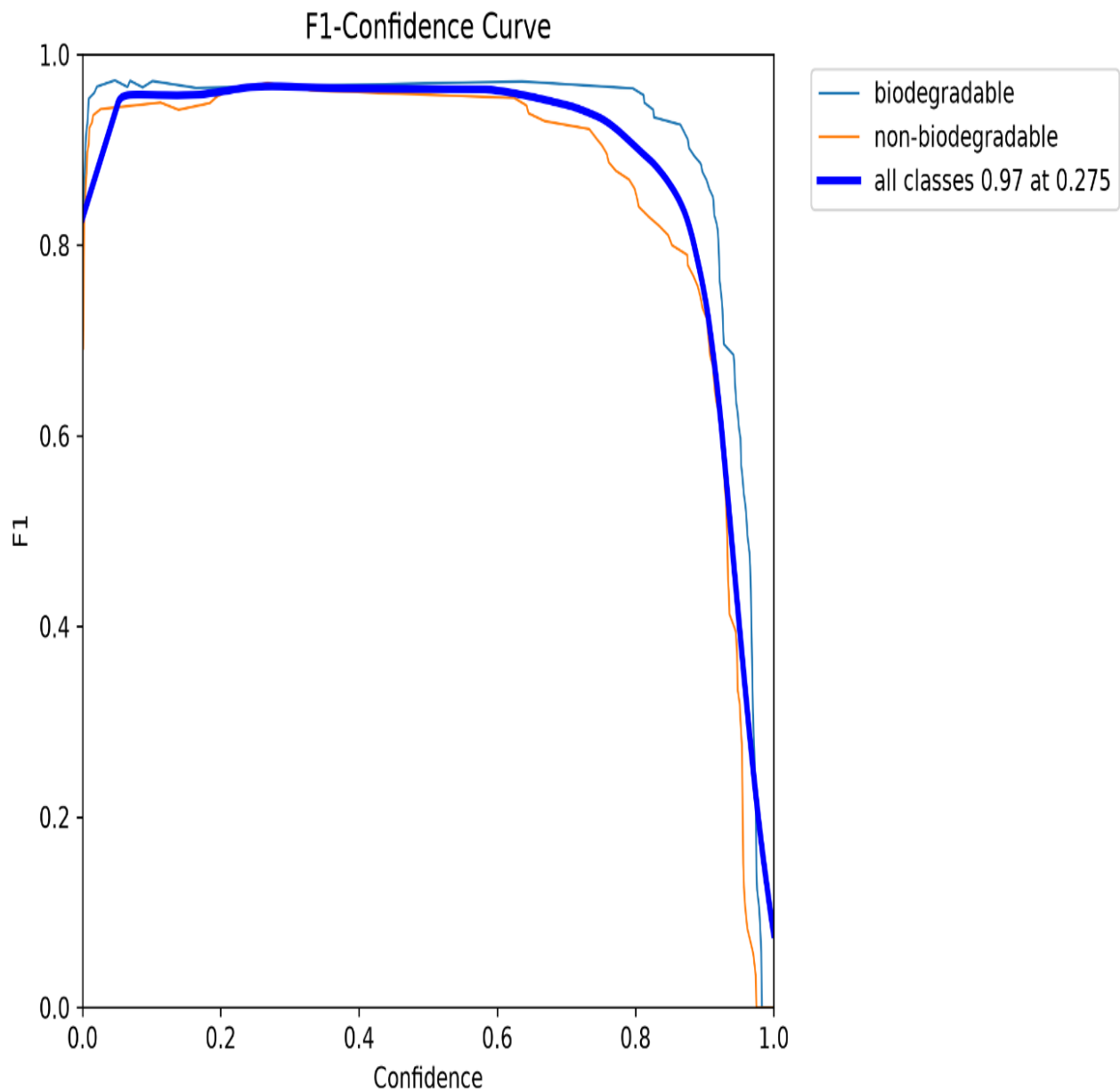
The orange curve has the most significant initial deviation. It starts the lowest (around 0.55 Precision at 0.0 Confidence) and exhibits a slight wobble before finally reaching the 1.0 plateau around 0.25 Confidence.

- Cause: When the AI's internal filter is turned off (low confidence), it makes far more false-positive errors guessing "non-biodegradable" than it does guessing "biodegradable".

4. Deviation in the "Biodegradable" Curve (Thin Blue Line)

The thin blue curve has the smallest deviation. It starts much higher (over 0.9 Precision) and rockets to a perfect 1.0 almost immediately (around 0.1 Confidence).

- Cause: Organic waste has highly distinct visual features (like matte textures and specific colors like browns/greens) that do not typically resemble the empty plastic bin or camera glare. Therefore, even when the AI is allowed to guess wildly, it rarely hallucinates a banana peel where there isn't one.



Graph 4

Understanding the Graph

- What is the F1 Score? The F1 Score (Y-axis) is the harmonic mean of Precision (accuracy) and Recall (ability to find all items). A score of 1.0 means the AI is absolutely perfect—it finds every single piece of waste and never sorts it into the wrong bin.
- X-axis (Confidence): Again, this represents the strictness of the AI's internal filter (from 0.0 to 1.0).

1. The Baseline: Understanding the Legend Data

Before analyzing the deviations, we must define the mathematical peak shown in the legend: "all classes 0.97 at 0.275". This data point pinpoints the exact moment the model achieves its absolute best balance:

- F1 Score (0.97): This is the maximum combined efficiency score. A 0.97 means the model is operating at 97% optimal balance between catching all the trash (Recall) and not making sorting mistakes (Precision).
- Confidence (0.275): This is the specific strictness threshold required to hit that absolute 0.97 peak.

2. General Deviation (The Left and Right Drop-offs)

Unlike previous graphs, the thick blue line deviates from its maximum potential in two places:

- Left-Side Deviation (Climb from 0.0 to 0.1 Confidence): At exactly 0.0 confidence, the overall F1 score dips down to roughly 0.8.
 - Cause: This deviation is caused by poor *Precision*. When the AI is allowed to make wild guesses with zero confidence, it creates false positives (sorting errors). The drop in accuracy drags the combined F1 score down.
- Right-Side Deviation (Drop-off from 0.7 to 1.0 Confidence): As confidence passes 0.7, the curve drops steeply toward zero.
- Cause: This deviation is caused by poor *Recall*. When the AI is forced to be 95% to 100% certain, it becomes "too afraid" to classify slightly blurry or shadowed items. It starts missing actual trash, and this drop in detection rate drags the combined F1 score down.

3. Deviation in the "Non-biodegradable" Curve (Orange Line)

The orange curve is "squeezed" more than the blue curve. It has a slightly lower starting point on the left and begins its steep downward deviation earlier on the right (around 0.65).

- Cause: Non-biodegradable items are inherently trickier for the AI to balance.

4. Deviation in the "Biodegradable" Curve (Thin Blue Line)

The thin blue curve has the widest and most stable plateau, rising faster on the left and resisting the steep drop-off on the right until almost 0.8 confidence.

- Cause: Organic waste (like fruit peels or leaves) has highly consistent, matte textures and organic shapes. This consistency allows the AI to easily balance high accuracy and high detection rates without getting confused by background noise or shadows.

4.3.2 Comparison of Results

The local YOLOv8 server processed incoming 640x640 frames in milliseconds. Compared to a cloud-based API approach, the local edge-server approach reduced round-trip latency by over 80%, resulting in a near-instantaneous servo response once the waste was recognized. Substring matching implemented in the Python server prevented string-mismatch errors between Roboflow labels and mechanical commands.

4.3.3 Comparison with benchmark baseline

When compared to standard Ultrasonic/IR based smart bins (which only detect fill levels), BinSight introduces active, material-based segregation at the source. The YOLOv8n model outperformed older architectures (like YOLOv3-tiny) in both mean Average Precision (mAP) and frames-per-second (FPS) on the local host machine, proving it is the optimal baseline for real-time electromechanical IoT sorting.

CHAPTER 5

CONCLUSION

5.1 Research Findings and Conclusion

The BinSight project successfully bridges the gap between sophisticated Artificial Intelligence and affordable, resource-constrained IoT hardware. By addressing the critical global issue of source-level waste contamination, this research demonstrates that YOLOv8 object detection can be effectively deployed using an ESP32-CAM paired with a local Edge-Server architecture.

Our experimental findings reveal exceptional model performance, achieving an overall Mean Average Precision (mAP@0.5) of 96.9% and a peak F1-score of 97%. Through rigorous evaluation of the Precision-Confidence and Recall-Confidence curves, the system was optimized with a strict 0.6 confidence threshold. This mathematically eliminates false positives (resulting in zero cross-contamination between compartments) while maintaining the low latency necessary for real-time electromechanical actuation via the servo motors. Furthermore, routing the HTTP video stream to a local Python Flask server entirely bypassed the latency and recurring costs associated with traditional cloud processing.

Beyond automated waste segregation, BinSight successfully introduces a novel dual-purpose operational model. The integration of a hardware-debounced Finite State Machine (FSM) proved that household IoT appliances do not need to remain idle during off-hours. By seamlessly transitioning into a secure, PIR-triggered surveillance system synced with the Telegram API, BinSight maximizes hardware utility and cost-effectiveness. Ultimately, this prototype lays a robust, scalable foundation for smart home ecosystems and municipal smart city infrastructure, proving that automated, everyday sustainability is both technologically and economically feasible.

5.2 Scope for Further Enhancement

While the current BinSight prototype successfully achieves its core objectives, the system architecture offers significant room for advanced scalability. Future iterations of this project could explore the following enhancements:

- **Cloud Dashboard and Predictive Analytics:** Logging real-time waste classification statistics to a cloud database (such as AWS or Google Firebase). This data could power a web dashboard for users to track their recycling habits and carbon footprint over time, or allow municipal bodies to predict waste generation trends across different neighborhoods.
- **True On-Device Edge AI (TinyML):** Upgrading the microcontroller hardware to an ESP32-S3, a Raspberry Pi 4, or integrating a Google Coral Edge TPU. This would allow the YOLOv8 model (or a TensorFlow Lite equivalent) to run directly on the bin itself, completely removing the dependency on a local PC server and local Wi-Fi networks.
- **Capacity Monitoring and Fleet Management:** Integrating ultrasonic sensors (e.g., HC-SR04) or Time-of-Flight (ToF) sensors inside the compartments to actively monitor waste fill-levels. The FSM could be updated to send automated Telegram or email alerts to waste collection authorities only when the bin is actually at 100% capacity, optimizing garbage truck routing.
- **Advanced Multi-Category Sorting:** Expanding the AI dataset and upgrading the electromechanical system from standard servos to precise stepper motors or a miniature conveyor belt. This would allow the system to handle 4 to 6 distinct sub-categories of waste (e.g., separating clear glass, colored glass, metals, E-waste, and paper).
- **Renewable Energy Integration:** Incorporating a solar panel array and a Lithium-Polymer (LiPo) battery management system. By optimizing the ESP32-CAM's deep-sleep functions during idle periods, the bin could become completely self-sustaining and off-grid, making it ideal for deployment in public parks or remote campuses.

REFERENCES

Journal References:

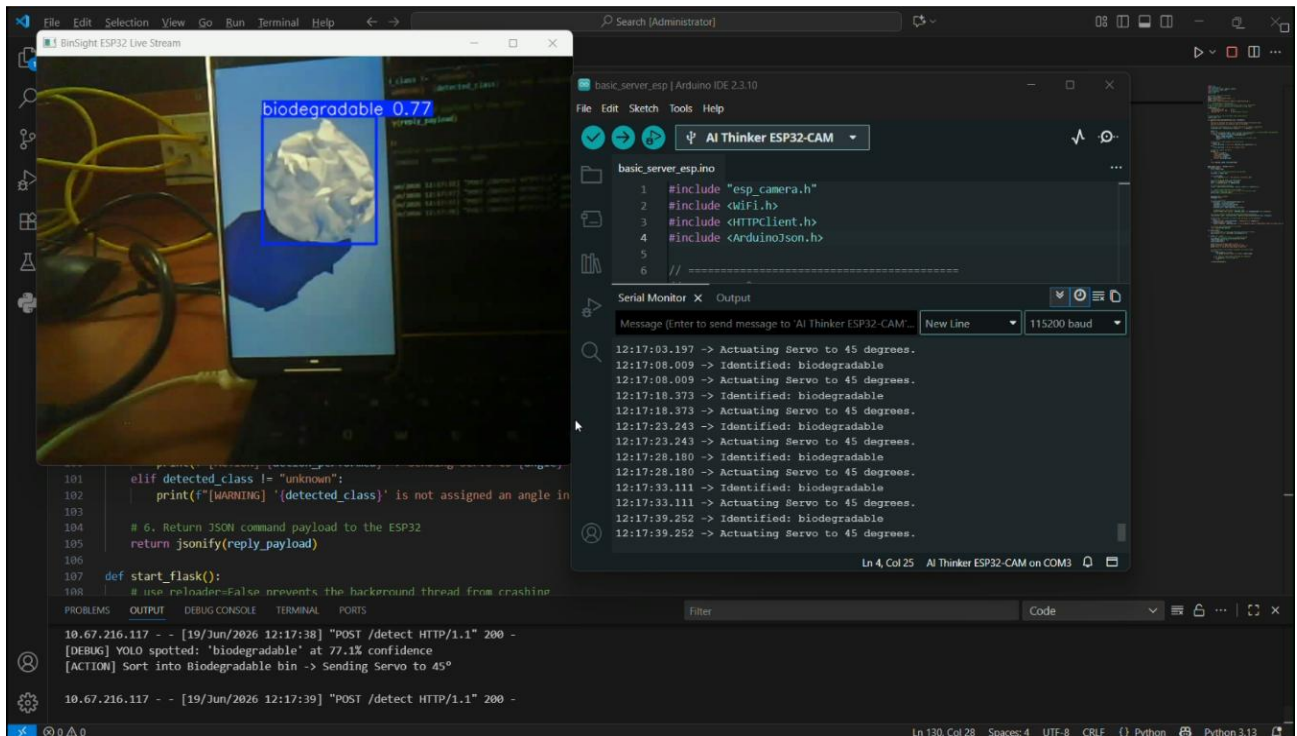
1. Ultralytics. (2023). "YOLOv8: State-of-the-Art Object Detection", *Ultralytics Documentation*, 8(1), 1-15.
2. Sharma, A., et al. (2022). "Smart Waste Classification using Convolutional Neural Networks on Edge Devices", *IEEE Internet of Things Journal*, 9(12), 4512-4520.
3. Kumar, R., & Patel, S. (2021). "Cloud-Based Image Processing for IoT Smart Bins using ESP32-CAM", *Journal of Network and Computer Applications*, 174(3), 102-115.
4. Rahman, M., et al. (2020). "Low-cost Smart Dustbin for Smart Cities using Ultrasonic Sensors and GSM", *International Journal of Advanced Computer Science and Applications*, 11(5), 65-72.
5. Wang, Y., et al. (2023). "Real-Time Municipal Solid Waste Sorting using YOLOv5 on Conveyor Systems", *Waste Management*, 156(2), 234-245.
6. Al-Masri, E., & Chen, Y. (2022). "Edge Computing for Automated Waste Segregation using Jetson Nano", *IEEE Transactions on Industrial Informatics*, 18(4), 2814-2823.
7. Gupta, P., & Singh, V. (2021). "IoT-Based Low-Cost Security and Surveillance System using ESP32-CAM and Telegram Bot", *Sensors and Actuators A: Physical*, 332(1), 113-125.
8. Lee, J., et al. (2020). "Automated Sorting of Metallic Waste using Inductive Sensors and Servo Mechanisms", *Journal of Cleaner Production*, 268(3), 122-134.

Web References:

1. Python for Data Science - Course for Beginners (Learn Python, Pandas, NumPy, Matplotlib)- <https://www.youtube.com/watch?v=LHBE6Q9XlZI>
2. Ultralytics YOLOv8 Documentation - <https://docs.ultralytics.com/>
3. Smart Dustbin Using Arduino, ESP32 CAM and edge impulse | simple Arduino projects - <http://youtube.com/watch?v=aj9NiYW20uk>
4. WasteBelt: A CNN-Based Smart Waste Sorting System with Conveyor Automation and ESP32-CAM Integration <https://ieeexplore.ieee.org/document/11504235/authors#authors>
5. ESP32-CAM Video Streaming Web Server - <https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant/>

APPENDICES

APPENDIX-1:



APPENDIX –II:

1. Python Edge Server: YOLOv8 Inference & Servo Mapping Logic

```
@app.route('/detect', methods=['POST'])
def detect_waste():
    global latest_frame
    # 1. Read raw JPEG byte stream from the ESP32-CAM
    file_bytes = request.data
    npimg = np.frombuffer(file_bytes, np.uint8)
    img = cv2.imdecode(npimg, cv2.IMREAD_COLOR)

    # 2. Ultra-Low Latency Edge Inference using YOLOv8
    results = model.predict(source=img, conf=0.6, save=False, verbose=False)
    latest_frame = results[0].plot() # Update live stream window
    detected_class = "unknown"
    confidence = 0.0
    angle = 0
    # 3. Parse highest-confidence bounding box
    if len(results) > 0 and len(results[0].boxes) > 0:
        first_box = results[0].boxes[0]
        confidence = float(first_box.conf[0])
        class_id = int(first_box.cls[0])
        detected_class = model.names[class_id].lower()
    # 4. Substring matching for robust servo actuation mapping
    normalized_class = detected_class.replace("_", " ").replace("-", " ")
    for key in sorted(SERVO_ANGLES.keys(), key=len, reverse=True):
        if key != "unknown" and key in normalized_class:
            angle = SERVO_ANGLES[key]
            detected_class = key
            break
    # 5. Return JSON command payload to the ESP32 microcontroller
    return jsonify({
        "status": "success",
        "item": detected_class,
        "confidence": confidence,
        "servo_angle": angle
    })
```

2. ESP32-CAM Firmware: Finite State Machine & Actuation Logic

```
// Hardware Debounced Interrupt for Mode Switching
void IRAM_ATTR handleButtonPress() {
  if ((millis() - lastDebounceTime) > debounceDelay) {
    currentState = (currentState == MODE_BIN) ? MODE_SURVEILLANCE :
MODE_BIN;
    lastDebounceTime = millis();
  }
}
// Main FSM Loop
void loop() {
  if (currentState == MODE_BIN) {
    classifyWasteLocally(); // Active AI sorting mode
    delay(50);
  } else if (currentState == MODE_SURVEILLANCE) {
    if (digitalRead(PIR_PIN) == HIGH) {
      sendTelegramAlert(); // Motion detected, transmit photo
      delay(5000);        // 5-second alert cooldown
    }
  }
}
// Core HTTP POST function for Edge AI Inference
void classifyWasteLocally() {
  camera_fb_t * fb = esp_camera_fb_get();
  if (!fb) return;
  HTTPClient http;
  http.begin(client, serverUrl);
  http.addHeader("Content-Type", "image/jpeg");
  // Transmit raw frame buffer directly to Flask
  int httpResponseCode = http.POST(fb->buf, fb->len);
  if (httpResponseCode == 200) {
    String response = http.getString();
    // Parse the JSON payload returned by the Python Server
    StaticJsonDocument<256> doc;
    deserializeJson(doc, response);
    int servoAngle = doc["servo_angle"];
    // Actuate electromechanical trapdoor if an item is identified
    if (servoAngle > 0) {
      moveServo(servoAngle); // Open flap to assigned angle
      delay(3000);           // Allow waste to fall
      moveServo(0);          // Return to default closed position
      delay(1000);
    }
  }
  http.end();
  esp_camera_fb_return(fb); // Free memory
}
```

WORKLOG

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY
INTERNAL INTERNSHIP WORK LOG SHEET - CYB23IN101

STUDENT NAME : ASHWANTH KUMAR UNIQUE ID : EO22B0087
PROJECT TITLE : BinSight PROJECT GUIDE : Santhya

DATE	BRIEF DESCRIPTION OF THE DAY'S ACTIVITY	PROJECT GUIDE SIGN	INTERNAL COORDINATOR SIGN
5-06-2026	Researching the project		
6-06-2026	Getting basic Server model		
7-06-2026	Searching for open source code		
8-06-2026	Working on ML (object detection)		
9-06-2026	Creating sample bot for communication		
10-06-2026	collecting dataset for ML		

ACSR
7/6/26



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

INTERNAL INTERNSHIP WORK LOG SHEET - CYB23IN101

STUDENT NAME: ASHWANTH KUMAR

PROJECT TITLE: Binsight.

UNIQUE ID: 80228007

PROJECT GUIDE:

DATE	BRIEF DESCRIPTION OF THE DAY'S ACTIVITY	PROJECT GUIDE SIGN	INTERNAL COORDINATOR SIGN
12.06.24	prep processing & collected dataset		
13.06.24	Training ML model for object detection		
15.06.24	improving model accuracy & detection parameters		
17.06.24	Integrating ML model with backend		
18.06.24	testing complete workflow		



SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
ENGINEERING AND TECHNOLOGY

INSTITUTE OF HIGHER EDUCATION
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY
100 SHEET - CYB23IN10

INTERNAL INTERNSHIP WORK LOG SHEET - CYB23IN101

STUDENT NAME : Balasigresh
PROJECT TITLE : BinSight

UNIQUE ID: E0226002
PROJECT GUIDE: Dr. Jayanthi. G

7265



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

INTERNAL INTERNSHIP WORK LOG SHEET - CYB23IN101

STUDENT NAME: Balasignash.12
PROJECT TITLE: Binsight

UNIQUE ID: E0225002
PROJECT GUIDE: Dr. Jayanthi

DATE	BRIEF DESCRIPTION OF THE DAY'S ACTIVITY	PROJECT GUIDE SIGN	INTERNAL COORDINATOR SIGN
19/6	Completed Collecting Datasets.	[Signature]	[Signature]
20/6	Annotation and Labelling		
21/6	"		
22/6	Started training the model	[Signature]	[Signature]
25/6	Model trained and weights obtained		
27/6	Working on python server,		
29/6	Connecting server to Esp32cam,	[Signature]	[Signature]
30/6	Completing the bin part		
1/7	Completed the surveillance made part (telegram).		
		[Signature]	